S.G. Ganesh

About the Java Overflow Bug

In this column, we'll discuss a common overflow bug in JDK, which surprisingly occurs in the widely used algorithms like binary search and mergesort in C-based languages.

How does one calculate the average of two integers, say *i* and *j*? Trivial you would say: it is *(i + j) / 2*. Mathematically, that's correct, but it can overflow when *i* and *j* are either very large or very small when using fixed-width integers in C-based languages (like Java). Many other languages like Lisp and Python do not have this problem. Avoiding overflow when using fixed-width integers is important, and many subtle bugs occur because of this problem.

In his popular blog post [1], Joshua Bloch (Java expert and author of books on Java intricacies) writes about how a bug [2] in binarySearch and mergeSort algorithms was found in his code in ***java.util.Arrays*** class in JDK. It read as follows:

```
1: public static int binarySearch(int[] a, int key) {
2:         int low = 0;
3:         int high = a.length - 1;
4:
5:         while (low <= high) {
6:                 int mid = (low + high) / 2;
7:                 int midVal = a[mid];
8:
9:                 if (midVal < key)
10:                        low = mid + 1
11:                else if (midVal > key)
12:                        high = mid - 1;
13:                else
14:                        return mid; // key found
15:        }
16:        return -(low + 1);  // key not found.
17: }
```

The bug is in line 6—"int mid = (low + high) / 2;". For large values of 'low' and 'high', the expression overflows and becomes a negative number (since 'low' and 'high' represent array indexes, they cannot be negative).

However, this bug is not really new—rather, it is usually not noticed. For example, the classic K & R book [3] on C has the same code (pg 52). For pointers, the expression *(low + mid) / 2* is wrong and will result in compiler error, since it is not possible to add two pointers. So, the book's solution is to use subtraction (pg 113):

```
mid = low + (high-low) / 2
```

This finds 'mid' when 'high' and 'low' are of the same sign (they are pointers, they can never be negative). This is also a solution for the overflow problem we discussed on Java.

Is there any other way to fix the problem? If 'low' and 'high' are converted to unsigned values and then divided by 2, it will not overflow, as in:

```
int mid = ( (unsigned int) low + (unsigned int) high) / 2;
```

But Java does not support unsigned numbers. Still, Java has an unsigned right shift operator (>>>)—it fills the right-most shifted bits with 0 (positive values remain as positive numbers; also known as 'value preserving'). For the Java right shift operator >>, the sign of the filled bit is the value of the sign bit (negative values remain negative and positive values remain positive; also known as 'sign-preserving'). Just as an aside for C/C++ programmers: C/C++ has only the >> operator and it can be sign or value preserving, depending on implementation. So we can use the >>> operator in Java:

```
int mid = (low + high) >>> 1;
```

The result of (low + high), when treated as unsigned values and right-shifted by 1, does not overflow!

Interestingly, there is another nice 'trick' to finding the average of two numbers: *(i & j) + (i ^ j) /2*. This expression looks strange, doesn't it? How do we get this expression? Hint: It is based on a well-known Boolean equality, for example, as noted in [4]: "(A AND B) + (A OR B) = A + B = (A XOR B) + 2 (A AND B)".

A related question: How do you detect overflow when adding two ***int***s? It's a very interesting topic and is the subject for next month's column. END

References:

- *googleresearch.blogspot.com/2006/06/extra-extra-read-all-about-it-nearly.html*
- *bugs.sun.com/bugdatabase/view_bug.do?bug_id=5045582*
- The C Programming Language, Brian W. Kernighan, Dennis M. Ritchie, Prentice-Hall, 1988.
- *home.pipeline.com/~hbaker1/hakmem/boolean.html#item23*

About the author:

S G Ganesh is a research engineer in Siemens (Corporate Technology). His latest book is "60 Tips on Object Oriented Programming", published by Tata McGraw-Hill in December 2007. You can reach him at sgganesh@gmail.com.